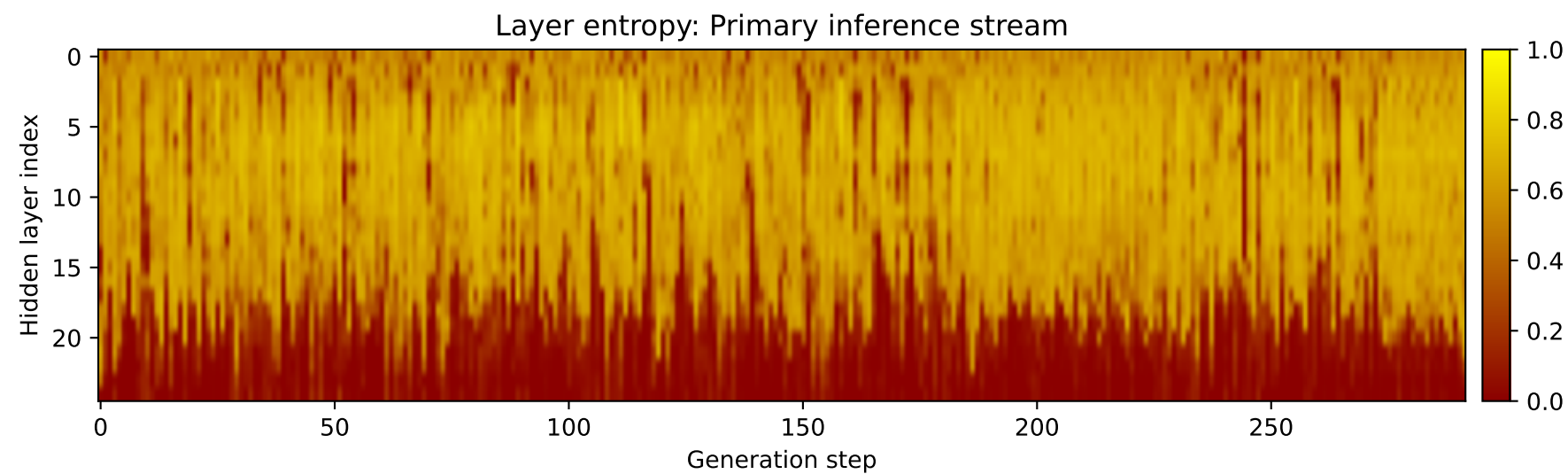
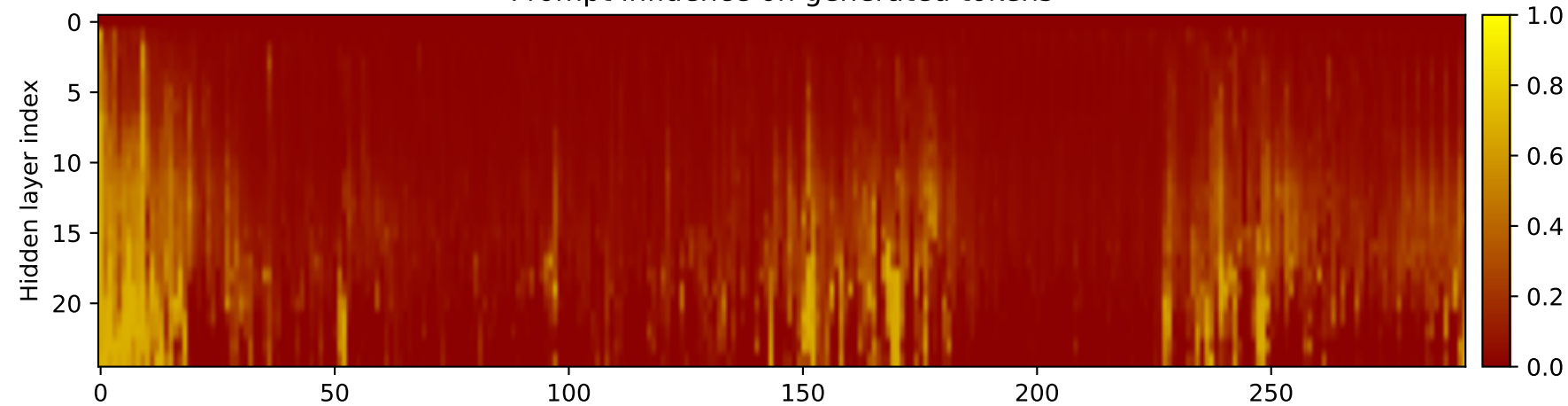
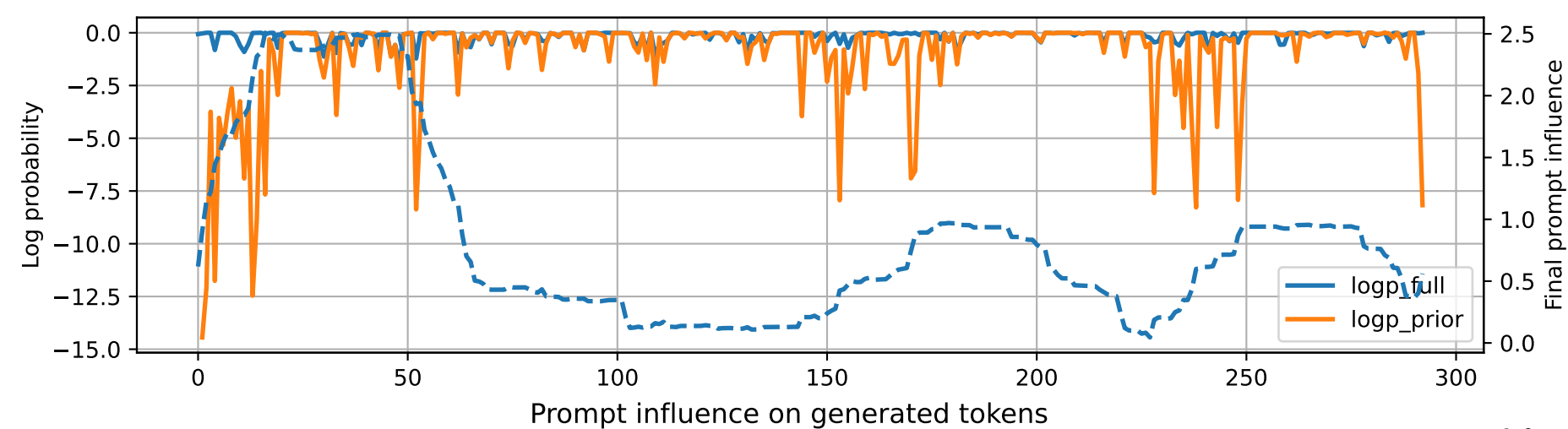


Prompt 1



PROMPT:
Task ID: PYC062 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC062 Your code must contain this required symbol:
def dfs_iterative Implement this task:
define function dfs_iterative(graph,start) returning visit
order.. Start your answer with the check-token
line, then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC062 # Define function dfs_iterative(graph,
start) def dfs_iterative(graph, start): #
Create an empty dictionary to store the visited nodes visited
= {} # Create a list to store the visit
order visit_order = [] # Define a recursive function to
traverse the graph def dfs(node):
If the node is the starting node, add it to the visit order
if node == start:
visit_order.append(node) # Otherwise, iterate over the
neighbors of the node for neighbor in
graph[node]: # If the neighbor is not visited,
recursively call dfs on it if neighbor
not in visited: dfs(neighbor) # Start the
recursive traversal from the starting node
dfs(start) # Return the visit order return visit_order
Example usage: graph = { 'A': ['B',
'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [], 'E':
[], 'F': [] } start_node = 'A' # Call
the dfs_iterative function with the graph and start node
visit_order = dfs_iterative(graph, start_node) #
Print the visit order print(visit_order) # Output: ['A', 'B',
'C', 'D', 'E', 'F']